DevOps Internship 2-Month Structured Guide

Week-wise concepts, required tools, installation commands, setup process, practical implementation, and weekly tasks

Program Objective

This guide restructures the DevOps internship manual into a clear week-wise format. Each week explains what interns need to learn, which tools they must install, how to install them, how to verify setup, what practical implementation they should do, and what weekly tasks they must submit. The purpose is to make interns learn DevOps step by step instead of jumping directly into tools without understanding the reason behind them.

Final Ability Expected After 2 Months

- Use Linux terminal confidently for file management, permissions, logs, processes, and basic scripting.
- Use Git and GitHub professionally with branches, commits, pull requests, and GitHub Actions.
- Build and run Docker containers, write Dockerfiles, and manage multi-service apps using Docker Compose.
- Understand PostgreSQL and MongoDB basics and connect databases with containerized applications.
- Deploy an application on AWS EC2 and understand S3 storage basics.
- Deploy containerized apps on Kubernetes using Minikube, kubectl, pods, deployments, and services.
- Set up monitoring using Prometheus and Grafana and understand basic alerting.
- Understand secrets management using Vault and why secrets should not be stored directly in code.
- Complete an end-to-end DevOps pipeline from GitHub to CI/CD, Docker, AWS deployment, monitoring, and secrets.

How Interns Should Use This Guide

- Do not install every tool on the first day. Install tools only when the week requires them.
- Before running commands, understand what the tool does and why it is used in DevOps.
- After installation, always run verification commands such as version checks and sample commands.
- Keep one GitHub repository for weekly submissions and documentation.
- Maintain a weekly README explaining commands used, screenshots, problems faced, and how they were solved.
- Practical implementation is not optional. DevOps is learned by doing, checking logs, fixing errors, and documenting steps.

8-Week Structured Roadmap

Week	Main Focus	Main Output
Week 1	Linux fundamentals and terminal confidence	Linux command practice, folder structure, scripts, logs, permissions
Week 2	Git, GitHub, and CI/CD foundation	GitHub repo, branches, PR workflow, first GitHub Actions pipeline

Week	Main Focus	Main Output
Week 3	Docker and Docker Compose	Dockerized sample app and multi-container compose setup
Week 4	Databases: PostgreSQL and MongoDB	Database containers, CRUD practice, app-database connection
Week 5	AWS deployment: EC2, SSH, S3	Dockerized app deployed on EC2 and sample S3 upload
Week 6	Kubernetes with Minikube	Local Kubernetes deployment, service exposure, scaling, logs
Week 7	Monitoring, Grafana, Prometheus, Vault	Dashboard, alerts concept, secrets stored and retrieved through Vault
Week 8	Final DevOps pipeline integration	End-to-end pipeline with docs, deployment, monitoring, and presentation

Week 1: Linux Fundamentals and Terminal Confidence

Main learning goal: The goal of Week 1 is to make interns comfortable with Linux because most servers, Docker containers, Kubernetes nodes, cloud VMs, and DevOps automation environments run on Linux. Interns should not only memorize commands; they should understand paths, files, permissions, processes, logs, and shell scripts.

Detailed Topic Explanation

Linux command line

Interns must understand that the terminal is the primary control surface for servers. Unlike GUI work, DevOps tasks are usually performed using commands over SSH. They should know absolute paths, relative paths, home directory, current directory, and command options.

File system structure

Important directories include /home for users, /etc for configuration, /var/log for logs, /tmp for temporary files, /usr/bin for installed binaries, and /opt for optional application files.

Permissions

Linux permissions control who can read, write, and execute files. Interns must understand user, group, others, and chmod numbers such as 755 and 644.

Processes and logs

DevOps engineers often check running processes and system logs when apps fail. Commands such as ps, top, htop, journalctl, tail, and grep help identify problems.

Bash scripting

Scripts automate repeated work. Interns should know variables, echo, conditions, loops, command substitution, and how to make a script executable.

Tools and Technologies Needed This Week

Tool / Tech	Why it is needed
Linux terminal	Use Ubuntu/Linux directly, WSL on Windows, or a Linux VM.
VS Code	Used for editing shell scripts and documentation.
Git Bash optional	Useful on Windows, but WSL is better for Linux practice.

Installation Commands and Setup Process

Windows WSL Ubuntu setup

```
# Open PowerShell as Administrator
wsl --install
wsl --install -d Ubuntu
wsl --status

# After restart, open Ubuntu and update packages
sudo apt update
sudo apt upgrade -y
```

Ubuntu package update and useful tools

```
sudo apt update
sudo apt install -y curl wget git tree htop unzip nano vim net-tools

# Verify tools
pwd
ls
```

```
whoami
git --version
tree --version
htop --version
```

macOS terminal preparation

```
# Install Homebrew from https://brew.sh first, then:
brew update
brew install git tree htop wget

# Verify
git --version
tree --version
```

Step-by-Step Setup Process

- Create a dedicated internship folder inside the home directory.
- Create subfolders for scripts, logs, notes, and weekly submissions.
- Practice every command inside this folder so interns do not damage system files.
- Write every command and its purpose in README.md.

Practical Implementation Explanation

Interns must create a Linux practice workspace, write shell scripts for system information and backup simulation, inspect logs, use grep to search text, and practice file permissions. The practical work should train them to use terminal confidently before cloud and Docker work starts.

Practical Commands / Lab Flow

```
mkdir -p ~/devops-internship/week1/{scripts,logs,notes,backup}
cd ~/devops-internship/week1
pwd
tree

echo "DevOps Linux Practice" > notes/intro.txt
cp notes/intro.txt backup/intro-copy.txt
mv backup/intro-copy.txt backup/intro-backup.txt
rm -i backup/intro-backup.txt

cat > scripts/system_info.sh <<'EOF'
#!/bin/bash
echo "User: $(whoami)"
echo "Date: $(date)"
echo "Current directory: $(pwd)"
echo "Disk usage:"
df -h
echo "Memory usage:"
free -h
EOF

chmod +x scripts/system_info.sh
./scripts/system_info.sh | tee logs/system-info.log

grep "User" logs/system-info.log
ps aux | head
top
```

Weekly Tasks and Deliverables

- Create Linux folder structure for the internship.
- Write system_info.sh and run it successfully.
- Create a sample log file and search it using grep.
- Practice chmod by making a script executable and non-executable.
- Submit README.md with commands, screenshots, and explanation.

Common Setup Problems and Fixes

Problem	Likely Fix
Permission denied	File is not executable. Use chmod +x filename.sh.
command not found	Tool is not installed or PATH is not loaded. Install package or reopen terminal.
No such file or directory	Path is wrong. Use pwd, ls, and tab completion to verify path.

Week 2: Git, GitHub, and CI/CD Foundation

Main learning goal: Week 2 teaches interns how DevOps teams manage source code and automate checks. Git stores history, GitHub enables collaboration, and CI/CD runs automated workflows when code is pushed or pull requests are opened.

Detailed Topic Explanation

Git basics

Git tracks changes through commits. Interns must understand working directory, staging area, commit history, branches, remotes, and conflicts.

GitHub workflow

Professional teams do not usually work directly on main. They create feature branches, push changes, open pull requests, discuss review comments, and merge after approval.

CI/CD meaning

Continuous Integration checks code automatically. Continuous Deployment/Delivery prepares or releases changes automatically. GitHub Actions is a common tool for this.

YAML workflow files

GitHub Actions workflows are written in YAML under .github/workflows. Interns must understand indentation, triggers, jobs, steps, and actions.

Tools and Technologies Needed This Week

Tool / Tech	Why it is needed
Git	Version control.
GitHub account	Remote repository and Actions.
GitHub CLI optional	Useful for command-line GitHub work.
VS Code	Source control and YAML editing.

Installation Commands and Setup Process

Ubuntu / WSL Git setup

```
sudo apt update
sudo apt install -y git

git --version
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
git config --global init.defaultBranch main

git config --list
```

GitHub CLI optional

```
# Ubuntu/WSL using snap if available
sudo snap install gh

# macOS
brew install gh

# Login
gh auth login
gh auth status
```

SSH key setup for GitHub

```
ssh-keygen -t ed25519 -C "your-email@example.com"
cat ~/.ssh/id_ed25519.pub

# Copy public key to GitHub: Settings -> SSH and GPG keys
ssh -T git@github.com
```

Step-by-Step Setup Process

- Create a GitHub repository named devops-internship-labs.
- Clone the repository locally or initialize local repository and connect remote.
- Create a week2 branch for Git and CI/CD practice.
- Add a GitHub Actions workflow that runs a simple shell command or script.

Practical Implementation Explanation

Interns must push Week 1 Linux scripts to GitHub, create a new branch, update README.md, open a pull request, and set up a simple GitHub Actions workflow that validates the repository by running shell commands.

Practical Commands / Lab Flow

```
cd ~/devops-internship

git init
git add .
git commit -m "chore: add week 1 linux practice"
git branch -M main
git remote add origin git@github.com:YOUR_USERNAME/devops-internship-labs.git
git push -u origin main

git checkout -b week2-git-cicd
mkdir -p .github/workflows
cat > .github/workflows/basic-check.yml <<'EOF'
name: Basic DevOps Check

on:
  push:
    branches: [ main, week2-git-cicd ]
  pull_request:
    branches: [ main ]

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Show repository files
        run: ls -la
      - name: Run shell syntax check
        run: bash -n week1/scripts/system_info.sh
EOF

git add .
git commit -m "ci: add basic github actions workflow"
git push -u origin week2-git-cicd
```

Weekly Tasks and Deliverables

- Push Week 1 work to GitHub.
- Create a feature branch for Week 2.
- Create a pull request and merge it after checking workflow result.
- Add GitHub Actions workflow under .github/workflows.
- Document CI/CD meaning and workflow screenshot in README.md.

Common Setup Problems and Fixes

Problem	Likely Fix
Authentication failed	Use SSH key or GitHub token. Password authentication does not work for Git operations.
YAML workflow not running	Check file path .github/workflows/*.yml and branch trigger.
Merge conflict	Open conflicted file, remove conflict markers, keep correct code, commit fix.

Week 3: Docker and Docker Compose

Main learning goal: Week 3 teaches containerization. Docker packages an application with its runtime and dependencies so it runs consistently across machines. Docker Compose runs multiple services together, such as app, database, and cache.

Detailed Topic Explanation

Image vs container

An image is a template. A container is a running instance of an image. Interns must understand build, run, stop, remove, logs, ports, volumes, and environment variables.

Dockerfile

A Dockerfile describes how to build an image: base image, working directory, copy files, install dependencies, expose ports, and command.

Docker Compose

Compose defines multiple containers in a YAML file. It is useful for local development with app, PostgreSQL, MongoDB, Redis, and other services.

Volumes and networking

Volumes persist data outside containers. Compose creates a default network so services can talk using service names.

Tools and Technologies Needed This Week

Tool / Tech	Why it is needed
Docker Engine or Docker Desktop	Container runtime.
Docker Compose plugin	Multi-container management.
A sample app	Can be Node, Python, or static nginx app.

Installation Commands and Setup Process

Ubuntu Docker Engine setup

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo   "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.co

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER
newgrp docker

docker --version
docker compose version
docker run hello-world
```

macOS / Windows Docker Desktop

```
Install Docker Desktop from https://docs.docker.com/desktop/ then run:

docker --version
docker compose version
docker run hello-world
```

Docker cleanup helpful commands

```
docker ps
docker ps -a
docker images
docker logs <container-name>
docker stop <container-name>
docker rm <container-name>
docker system df
```

Step-by-Step Setup Process

- Verify Docker works using hello-world.
- Create a simple app folder and Dockerfile.
- Build image locally.
- Run container with port mapping.
- Write docker-compose.yml to run app with one supporting service.

Practical Implementation Explanation

Interns must create a simple containerized application, build an image, run it as a container, check logs, stop/remove containers, and then use Docker Compose to manage services together. The main learning is understanding what happens during build and run.

Practical Commands / Lab Flow

```
mkdir -p ~/devops-internship/week3/docker-app
cd ~/devops-internship/week3/docker-app

cat > index.html <<'EOF'
<h1>DevOps Docker App</h1>
<p>Running inside a container.</p>
EOF

cat > Dockerfile <<'EOF'
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
EOF

docker build -t devops-docker-app:v1 .
docker run -d --name devops-web -p 8080:80 devops-docker-app:v1
curl http://localhost:8080
docker logs devops-web
docker stop devops-web
docker rm devops-web

cat > docker-compose.yml <<'EOF'
services:
  web:
    build: .
    ports:
      - "8080:80"
EOF

docker compose up -d
docker compose ps
docker compose logs
docker compose down
```

Weekly Tasks and Deliverables

- Install Docker and verify hello-world.
- Build one custom Docker image using Dockerfile.
- Run the image as a container with port mapping.
- Use docker logs, docker ps, docker stop, and docker rm.
- Create docker-compose.yml and run docker compose up/down.
- Document image, container, port, volume, and compose concepts.

Common Setup Problems and Fixes

Problem	Likely Fix
Cannot connect to Docker daemon	Docker service/Desktop is not running or user permission is missing.
Port already allocated	Another process uses the port. Change host port or stop old container.
Build fails	Check Dockerfile path, base image, and copied files.

Week 4: Databases - PostgreSQL and MongoDB

Main learning goal: Week 4 teaches how applications store data. PostgreSQL is relational and structured. MongoDB is document-based and flexible. DevOps interns should know how to run databases locally, manage credentials, persist data, check logs, connect applications, and troubleshoot database containers.

Detailed Topic Explanation

PostgreSQL

A relational database using tables, rows, columns, primary keys, foreign keys, and SQL queries. Common in production systems.

MongoDB

A document database storing JSON-like documents inside collections. Useful when data shape is flexible.

Database containers

Databases in Docker require environment variables, ports, and volumes so data persists after container restart.

Connection strings

Apps connect using URLs such as postgres://user:pass@host:5432/db and mongodb://user:pass@host:27017.

Backup and data persistence

DevOps must understand that deleting containers is safe only if data is stored in volumes or backups.

Tools and Technologies Needed This Week

Tool / Tech	Why it is needed
PostgreSQL container/client	SQL database.
MongoDB container/client	NoSQL database.
Docker Compose	Run databases together.
pgAdmin/Compass optional	GUI tools for learning.

Installation Commands and Setup Process

Database clients on Ubuntu / WSL

```
sudo apt update
sudo apt install -y postgresql-client

# MongoDB shell can be installed from MongoDB docs, or use container exec.
psql --version
```

Run PostgreSQL using Docker

```
docker run -d --name devops-postgres   -e POSTGRES_USER=devops   -e POSTGRES_PASSWORD=devopspass   -e POSTGRES_DB

docker exec -it devops-postgres psql -U devops -d devopsdb
```

Run MongoDB using Docker

```
docker run -d --name devops-mongo   -e MONGO_INITDB_ROOT_USERNAME=devops   -e MONGO_INITDB_ROOT_PASSWORD=devopspa

docker exec -it devops-mongo mongosh -u devops -p devopspass --authenticationDatabase admin
```

Step-by-Step Setup Process

- Start PostgreSQL and MongoDB using Docker or Docker Compose.

- Create sample table in PostgreSQL and insert data.
- Create sample database and collection in MongoDB and insert documents.
- Check database container logs.
- Document connection strings and explain host, port, username, password, database.

Practical Implementation Explanation

Interns must run PostgreSQL and MongoDB in containers, perform basic CRUD operations, create persistent volumes, inspect logs, and write connection details as environment variables. They should understand SQL vs NoSQL from operational and application perspectives.

Practical Commands / Lab Flow

```
mkdir -p ~/devops-internship/week4/databases
cd ~/devops-internship/week4/databases

cat > docker-compose.yml <<'EOF'
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: devops
      POSTGRES_PASSWORD: devopspass
      POSTGRES_DB: devopsdb
    ports:
      - "5432:5432"
    volumes:
      - pg_data:/var/lib/postgresql/data

  mongo:
    image: mongo:7
    environment:
      MONGO_INITDB_ROOT_USERNAME: devops
      MONGO_INITDB_ROOT_PASSWORD: devopspass
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db

volumes:
  pg_data:
  mongo_data:
EOF

docker compose up -d
docker compose ps

docker exec -it databases-postgres-1 psql -U devops -d devopsdb -c "CREATE TABLE IF NOT EXISTS interns(id SERIAL
docker exec -it databases-postgres-1 psql -U devops -d devopsdb -c "INSERT INTO interns(name, track) VALUES ('Sam
docker exec -it databases-postgres-1 psql -U devops -d devopsdb -c "SELECT * FROM interns;"

docker exec -it databases-mongo-1 mongosh -u devops -p devopspass --authenticationDatabase admin --eval 'db.getSi
docker exec -it databases-mongo-1 mongosh -u devops -p devopspass --authenticationDatabase admin --eval 'db.getSi

docker compose logs postgres
docker compose logs mongo
```

Weekly Tasks and Deliverables

- Run PostgreSQL and MongoDB using Docker Compose.
- Create one PostgreSQL table and insert/select/update/delete data.
- Create one MongoDB collection and insert/find/update/delete documents.
- Explain SQL vs NoSQL in README.md.
- Explain why volumes are needed for databases.
- Add database connection strings to a sample .env.example file.

Common Setup Problems and Fixes

Problem	Likely Fix
password authentication failed	Username/password/database name do not match environment variables.
container name different	Compose prefixes service names with folder name. Use docker compose ps to find actual name.
data lost after down -v	docker compose down -v deletes volumes. Use carefully.

Week 5: AWS Deployment - EC2, SSH, Docker on Server, and S3

Main learning goal: Week 5 moves interns from local machine to cloud server. AWS EC2 is used as a virtual machine for deployment. Interns learn SSH, security groups, Docker installation on EC2, running containers on a server, and S3 object storage basics.

Detailed Topic Explanation

EC2

A cloud virtual machine. Interns should understand AMI, instance type, key pair, public IP, security group, and inbound rules.

SSH

Secure Shell allows remote terminal access to Linux servers using a private key.

Security group

A virtual firewall. Ports such as 22 for SSH, 80 for HTTP, 443 for HTTPS, and app ports must be allowed carefully.

Deployment using Docker

Instead of installing every app dependency manually, deploy Docker images or Docker Compose files on EC2.

S3

Object storage for files, backups, static assets, and logs. Interns should understand bucket, object, region, and permissions.

Tools and Technologies Needed This Week

Tool / Tech	Why it is needed
AWS account	Cloud infrastructure.
AWS CLI	Command-line AWS management.
SSH client	Remote server access.
Docker on EC2	Run containerized app.

Installation Commands and Setup Process

AWS CLI install on Linux x86_64

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version

# Configure credentials only if provided by trainer/admin
aws configure
```

SSH key permissions

```
# Example private key downloaded from AWS: devops-key.pem
chmod 400 devops-key.pem
ssh -i devops-key.pem ubuntu@YOUR_EC2_PUBLIC_IP
```

Docker install on EC2 Ubuntu

```
sudo apt update
sudo apt install -y docker.io docker-compose-v2
```

```
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker ubuntu
newgrp docker

docker --version
docker compose version
```

Step-by-Step Setup Process

- Create EC2 instance using Ubuntu LTS.
- Download key pair and keep it safe.
- Open required inbound ports in security group.
- SSH into EC2.
- Install Docker on EC2.
- Pull or copy Docker app from Week 3 and run it on EC2.
- Create S3 bucket and upload one sample file using AWS Console or CLI.

Practical Implementation Explanation

Interns must deploy a Dockerized app to EC2. They should connect through SSH, install Docker, run the container, expose the correct port, check logs, and access the app from browser using EC2 public IP. They also upload a sample file to S3 and explain how storage is separate from compute.

Practical Commands / Lab Flow

```
# From local machine
chmod 400 devops-key.pem
ssh -i devops-key.pem ubuntu@YOUR_EC2_PUBLIC_IP

# On EC2
sudo apt update
sudo apt install -y docker.io docker-compose-v2 git
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker ubuntu
newgrp docker

git clone https://github.com/YOUR_USERNAME/devops-internship-labs.git
cd devops-internship-labs/week3/docker-app

docker build -t devops-web-cloud:v1 .
docker run -d --name devops-web-cloud -p 80:80 devops-web-cloud:v1
docker ps
docker logs devops-web-cloud

# From local browser: http://YOUR_EC2_PUBLIC_IP

# S3 example after aws configure
aws s3 mb s3://your-unique-devops-intern-bucket
aws s3 cp README.md s3://your-unique-devops-intern-bucket/README.md
aws s3 ls s3://your-unique-devops-intern-bucket/
```

Weekly Tasks and Deliverables

- Launch Ubuntu EC2 instance.
- Connect using SSH successfully.
- Install Docker on EC2.
- Deploy Week 3 Docker app on EC2.
- Configure security group so app is reachable.
- Upload one file to S3 and document steps.
- Submit screenshots of EC2, terminal, running container, browser output, and S3 object.

Common Setup Problems and Fixes

Problem	Likely Fix
SSH timeout	Security group may not allow port 22 from your IP or instance is not running.
Permission denied publickey	Wrong key file, wrong username, or key permission not chmod 400.
App not opening in browser	Security group app port is closed or container is not running.

Week 6: Kubernetes with Minikube and kubectl

Main learning goal: Week 6 teaches container orchestration. Kubernetes runs and manages containers using pods, deployments, services, replicas, scaling, and self-healing. Minikube provides a local Kubernetes cluster for learning.

Detailed Topic Explanation

Cluster

A Kubernetes environment with control plane and worker node(s). Minikube creates a local single-node cluster.

Pod

Smallest deployable unit in Kubernetes. Usually contains one application container.

Deployment

Manages replicas of pods and supports rolling updates.

Service

Gives stable network access to pods because pod IPs can change.

kubectl

Command-line tool to interact with Kubernetes cluster.

YAML manifests

Kubernetes resources are usually defined in YAML files.

Tools and Technologies Needed This Week

Tool / Tech	Why it is needed
kubectl	Kubernetes command-line tool.
Minikube	Local Kubernetes cluster.
Docker	Container runtime for Minikube.
Existing Docker image	App image to deploy.

Installation Commands and Setup Process

Install kubectl on Ubuntu / WSL

```
sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kube
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /
sudo apt update
sudo apt install -y kubectl
kubectl version --client
```

Install Minikube on Linux x86_64

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
minikube start --driver=docker
kubectl get nodes
```

macOS install with Homebrew

```
brew install kubectl minikube
minikube start
```

```
kubectl get nodes
```

Step-by-Step Setup Process

- Install Docker, kubectl, and Minikube.
- Start Minikube cluster.
- Create deployment.yaml and service.yaml.
- Apply manifests using kubectl apply.
- Check pods, deployments, and services.
- Expose app using minikube service or port-forward.
- Scale deployment and observe new pods.

Practical Implementation Explanation

Interns must deploy their Docker app to local Kubernetes. They should create YAML manifests, apply them, inspect pods, check logs, expose the service, scale replicas, delete pods, and observe Kubernetes recreating them.

Practical Commands / Lab Flow

```
mkdir -p ~/devops-internship/week6/kubernetes
cd ~/devops-internship/week6/kubernetes

minikube start --driver=docker
kubectl get nodes

cat > deployment.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: devops-web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: devops-web
  template:
    metadata:
      labels:
        app: devops-web
    spec:
      containers:
        - name: devops-web
          image: nginx:alpine
          ports:
            - containerPort: 80
EOF

cat > service.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: devops-web-service
spec:
  type: NodePort
  selector:
    app: devops-web
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
EOF

kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods
kubectl get deployments
kubectl get services
kubectl logs deployment/devops-web
kubectl scale deployment devops-web --replicas=3
kubectl get pods
minikube service devops-web-service
```

```
# Cleanup if needed
kubectl delete -f service.yaml
kubectl delete -f deployment.yaml
```

Weekly Tasks and Deliverables

- Install Minikube and kubectl.
- Start a local Kubernetes cluster.
- Deploy an app using Deployment YAML.
- Expose the app using Service YAML.
- Scale pods from 2 to 3 replicas.
- Check logs and describe resources.
- Document pods, deployments, services, replicas, and NodePort.

Common Setup Problems and Fixes

Problem	Likely Fix
minikube start fails	Docker may not be running or virtualization is disabled.
ImagePullBackOff	Image name is wrong or private registry auth is missing.
Service not reachable	Check service type, selector labels, port, targetPort, and minikube tunnel/service command.

Week 7: Monitoring, Grafana, Prometheus, Alerts, and Vault Secrets

Main learning goal: Week 7 teaches observability and secrets management. Monitoring shows whether systems are healthy. Prometheus collects metrics, Grafana visualizes them, alerts notify problems, and Vault stores sensitive values securely.

Detailed Topic Explanation

Monitoring

Monitoring tracks system health, CPU, memory, disk, request rate, errors, and app availability.

Prometheus

A metrics collection and time-series database tool. It scrapes targets on configured intervals.

Grafana

A dashboard and visualization tool that reads data from Prometheus and other sources.

Alerting

Alerts notify teams when metrics cross dangerous thresholds.

Vault

Secrets management tool for tokens, passwords, API keys, certificates, and other sensitive data.

Secrets best practice

Secrets should not be committed to GitHub, hardcoded in app files, or shared in screenshots.

Tools and Technologies Needed This Week

Tool / Tech	Why it is needed
Prometheus	Metrics collection.
Grafana	Dashboards.
Node Exporter optional	Host metrics.
Vault	Secrets management.
Docker Compose	Easy local setup.

Installation Commands and Setup Process

Run Prometheus and Grafana with Docker Compose

```
mkdir -p ~/devops-internship/week7/monitoring
cd ~/devops-internship/week7/monitoring

cat > prometheus.yml <<'EOF'
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["prometheus:9090"]
EOF

cat > docker-compose.yml <<'EOF'
```

```
services:
  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin
EOF

docker compose up -d
# Prometheus: http://localhost:9090
# Grafana: http://localhost:3000
```

Run Vault in dev mode using Docker

```
docker run -d --name devops-vault   --cap-add=IPC_LOCK   -e VAULT_DEV_ROOT_TOKEN_ID=root   -e VAULT_DEV_LISTEN_AD

export VAULT_ADDR=http://127.0.0.1:8200
export VAULT_TOKEN=root
docker exec -e VAULT_ADDR=http://127.0.0.1:8200 -e VAULT_TOKEN=root devops-vault vault status
```

Vault secret commands

```
docker exec -e VAULT_ADDR=http://127.0.0.1:8200 -e VAULT_TOKEN=root devops-vault vault kv put secret/devops usern
docker exec -e VAULT_ADDR=http://127.0.0.1:8200 -e VAULT_TOKEN=root devops-vault vault kv get secret/devops
```

Step-by-Step Setup Process

- Run Prometheus and Grafana locally using Docker Compose.
- Open Grafana and add Prometheus as data source.
- Create a basic dashboard panel using Prometheus metrics.
- Run Vault dev server for learning only.
- Store and retrieve a sample secret from Vault.
- Document why Vault dev mode is not production setup.

Practical Implementation Explanation

Interns must run Prometheus, Grafana, and Vault locally. They should create a Grafana dashboard, connect Prometheus as a data source, explore metrics, and store/retrieve a secret from Vault. The focus is understanding observability and secure secret handling.

Practical Commands / Lab Flow

```
# After docker compose up -d for Prometheus/Grafana
curl http://localhost:9090/-/healthy
curl http://localhost:3000

docker compose ps
docker compose logs prometheus
docker compose logs grafana

# Grafana setup in browser:
# 1. Login admin/admin
# 2. Add data source: Prometheus
# 3. URL: http://prometheus:9090
# 4. Save and test
# 5. Create dashboard panel with query: up

# Vault secret practice
docker exec -e VAULT_ADDR=http://127.0.0.1:8200 -e VAULT_TOKEN=root devops-vault vault kv put secret/app DB_PASSW

docker exec -e VAULT_ADDR=http://127.0.0.1:8200 -e VAULT_TOKEN=root devops-vault vault kv get secret/app
```

Weekly Tasks and Deliverables

- Run Prometheus and Grafana using Docker Compose.
- Add Prometheus data source in Grafana.
- Create one dashboard panel using the up metric.
- Run Vault in dev mode using Docker.
- Store and retrieve one secret using Vault CLI.
- Write notes explaining metrics, dashboard, alert, and secret management.

Common Setup Problems and Fixes

Problem	Likely Fix
Grafana cannot connect to Prometheus	Use http://prometheus:9090 inside Compose network, not localhost from Grafana container.
Vault permission denied	VAULT_TOKEN may be missing or wrong.
Port 3000 already used	Change Grafana host port or stop existing service.

Week 8: Final DevOps Pipeline Integration

Main learning goal: Week 8 integrates everything learned. Interns must combine Linux, GitHub, CI/CD, Docker, databases, AWS, Kubernetes basics, monitoring, and secrets into one documented final DevOps workflow.

Detailed Topic Explanation

End-to-end DevOps flow

Code moves from local machine to GitHub, CI checks run, Docker image is built, app is deployed, logs are checked, monitoring is configured, and secrets are managed safely.

Documentation

A DevOps project is incomplete without clear setup, architecture, commands, environment variables, troubleshooting, and screenshots.

Operational thinking

Interns must be able to explain how to recover if deployment fails, container crashes, database is unavailable, or metrics are missing.

Final presentation

The intern should present architecture, tools used, setup commands, pipeline flow, deployment result, monitoring dashboard, and lessons learned.

Tools and Technologies Needed This Week

Tool / Tech	Why it is needed
All previous tools	Linux, Git, GitHub Actions, Docker, PostgreSQL/MongoDB, AWS, Kubernetes, Prometheus, Grafana, Vault.
README documentation	Final submission quality.
Screenshots	Proof of successful work.

Installation Commands and Setup Process

No new tool required

```
Week 8 should not introduce a major new tool. Interns should stabilize and integrate tools already installed in W
```

Final verification commands

```
git status
git log --oneline --max-count=5

docker --version
docker compose version
docker ps

kubectl get nodes
kubectl get pods

aws --version
aws sts get-caller-identity

curl http://YOUR_EC2_PUBLIC_IP
curl http://localhost:9090/-/healthy
```

Step-by-Step Setup Process

- Organize repository into week folders and final-project folder.

- Prepare final README.md with architecture and setup instructions.
- Ensure GitHub Actions workflow runs successfully.
- Ensure Docker app builds successfully.
- Ensure deployment is accessible from browser or documented as deployment-ready.
- Attach screenshots of CI/CD, Docker, AWS, monitoring, and secrets.

Practical Implementation Explanation

Interns must complete one final integrated DevOps submission. The final project should show that they can take an application from code to container, run checks through GitHub Actions, deploy to AWS EC2, connect data services where needed, add monitoring, and manage secrets safely.

Practical Commands / Lab Flow

```
mkdir -p ~/devops-internship/final-project/{docs,screenshots,deploy,monitoring,secrets}
cd ~/devops-internship

git checkout -b final-devops-pipeline

# Check Docker build
docker build -t final-devops-app:v1 week3/docker-app

# Optional compose validation
docker compose -f week4/databases/docker-compose.yml config

# Commit final documentation
git add .
git commit -m "docs: add final devops pipeline submission"
git push -u origin final-devops-pipeline
```

Weekly Tasks and Deliverables

- Submit GitHub repository with organized folders.
- Show working GitHub Actions pipeline.
- Show Dockerfile and docker-compose.yml.
- Show EC2 deployment or deployment-ready proof.
- Show PostgreSQL/MongoDB database setup notes.
- Show Prometheus/Grafana dashboard screenshot.
- Show Vault secret storage screenshot without exposing real secrets.
- Submit final README with architecture, setup steps, commands, troubleshooting, and learning reflection.

Common Setup Problems and Fixes

Problem	Likely Fix
Final repo is confusing	Use clear folder names and a root README with links to each week.
Secrets exposed	Remove secrets from Git history where possible and rotate exposed credentials.
Final demo fails	Prepare verification commands and screenshots before presentation.

Final Repository Structure Recommendation

A structured repository helps trainers review work quickly and helps interns learn professional DevOps documentation habits.

```
devops-internship-labs/
  README.md
  week1-linux/
    scripts/
    logs/
    README.md
  week2-git-cicd/
    .github/workflows/
    README.md
  week3-docker/
    Dockerfile
    docker-compose.yml
    README.md
  week4-databases/
    docker-compose.yml
    .env.example
    README.md
  week5-aws-deployment/
    deploy-notes.md
    screenshots/
  week6-kubernetes/
    deployment.yaml
    service.yaml
    README.md
  week7-monitoring-security/
    prometheus.yml
    docker-compose.yml
    vault-notes.md
  final-project/
    architecture.md
    setup.md
    troubleshooting.md
    screenshots/
```

Final README Checklist

- Project overview and goal.
- Tools used and why each tool was used.
- Week-wise learning summary.
- Installation commands and verification commands.
- Docker build and run commands.
- CI/CD workflow explanation.
- AWS deployment steps and security group explanation.
- Kubernetes YAML explanation.
- Monitoring dashboard screenshot and explanation.
- Vault secret management explanation without exposing real secrets.
- Common errors faced and how they were fixed.
- Final learning reflection.

Recommended Official References

- Docker documentation: https://docs.docker.com/
- Kubernetes documentation: https://kubernetes.io/docs/
- Minikube documentation: https://minikube.sigs.k8s.io/docs/

- Git documentation: https://git-scm.com/doc
- GitHub Actions documentation: https://docs.github.com/actions
- AWS documentation: https://docs.aws.amazon.com/
- Prometheus documentation: https://prometheus.io/docs/
- Grafana documentation: https://grafana.com/docs/
- HashiCorp Vault documentation: https://developer.hashicorp.com/vault/docs
- PostgreSQL documentation: https://www.postgresql.org/docs/
- MongoDB documentation: https://www.mongodb.com/docs/

Important Note for Trainers

This guide is designed to be clear and structured. It does not only list tasks. It explains what interns should learn, what tools are required in each week, how to install those tools, how to verify setup, what practical work should be completed, and what weekly deliverables should be submitted. This makes the DevOps internship easier to follow and easier to evaluate without needing a separate assessment section.